ECA 1011 – MICROPROCESSOR AND MICROCONTROLLERS

Name : Anbalagan P V

Reg No : 192524459

Experiment 02: Analysis, Development, and Verification of Logical, Bit Manipulation, and Shift Operations Using Assembly Language

Objectives

1. To analyze and verify 8-bit and 16-bit Logical operations (AND, OR, XOR) using an 8-bit microprocessor.
2. To analyze and verify Bit Manipulation operations (Setting, Clearing, and Complementing specific bits).
3. To analyze and verify Shift and Rotation operations (Logical/Arithmetic shifts left and right) using 8-bit data.

Components Required

- 8085 Simulator Software

Procedure

1. Launch the 8085 Simulator application on your system.
2. Type or paste the respective Assembly Language Program (ALP) into the editor/assembler window.
3. Assemble/Compile the code to convert the assembly mnemonics into machine hex code. ensure zero syntax errors.
4. Open the Memory Editor and manually input your test operands into the designated starting memory addresses.
5. Execute the program using the Run or Step-by-Step (Trace) feature to observe how registers and condition flags alter.
6. Verify your outputs by checking the target memory locations and register statuses against your theoretical manual calculations.

1. **To analyze and verify 8-bit and 16-bit Logical operations (AND, OR, XOR) using an 8-bit microprocessor.**

Code:

AND(8 bit):

LDA 4600

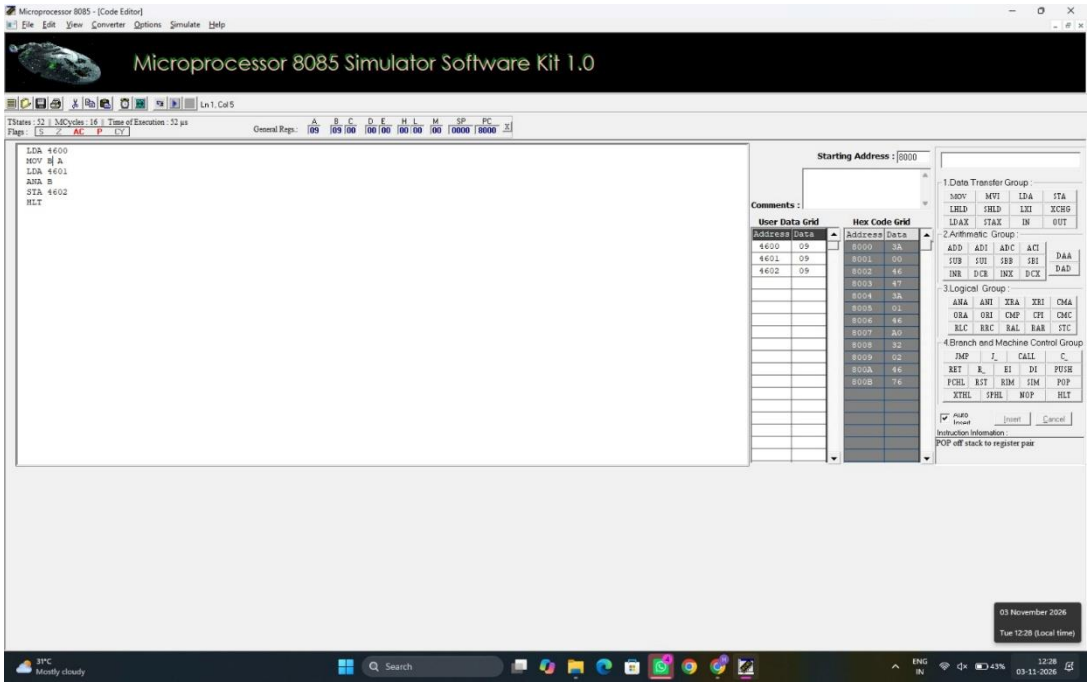
MOV B, A

LDA 4601

ANA B

STA 4602

HLT



Data Table:

4600	**09**
4601	**09**
4602	**09**

Code:

OR(8 bit):

LDA 4600

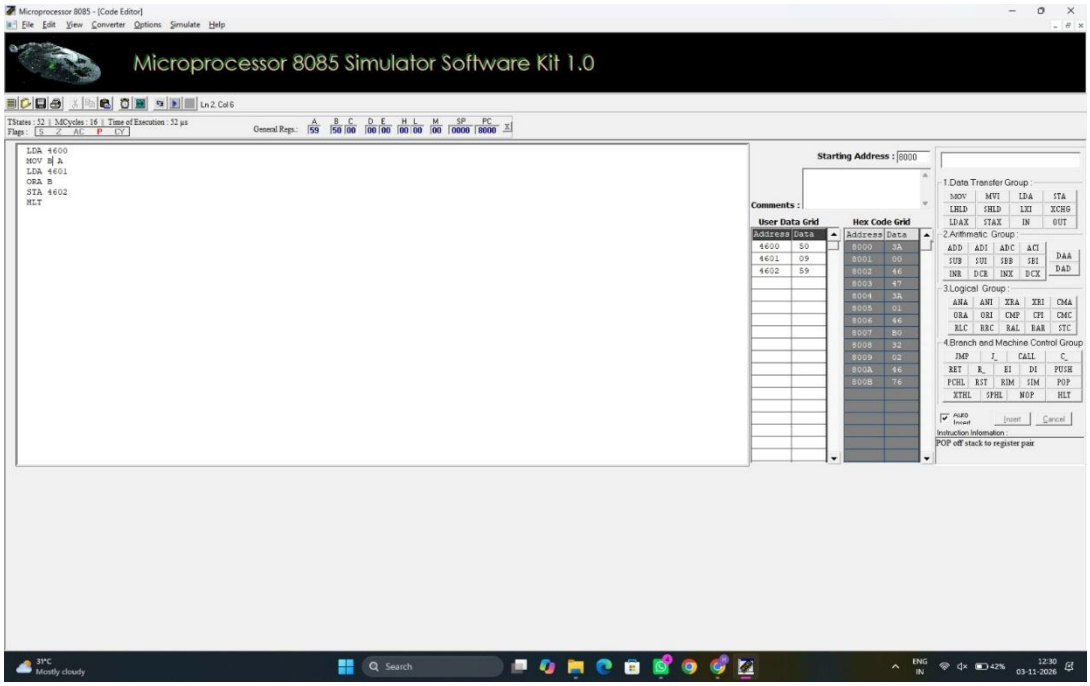
MOV B, A

LDA 4601

ORA B

STA 4602

HLT



Data Table:

4000	**59**
4001	**09**
4002	**59**

Code:

XOR(8 bit):

LDA 4600

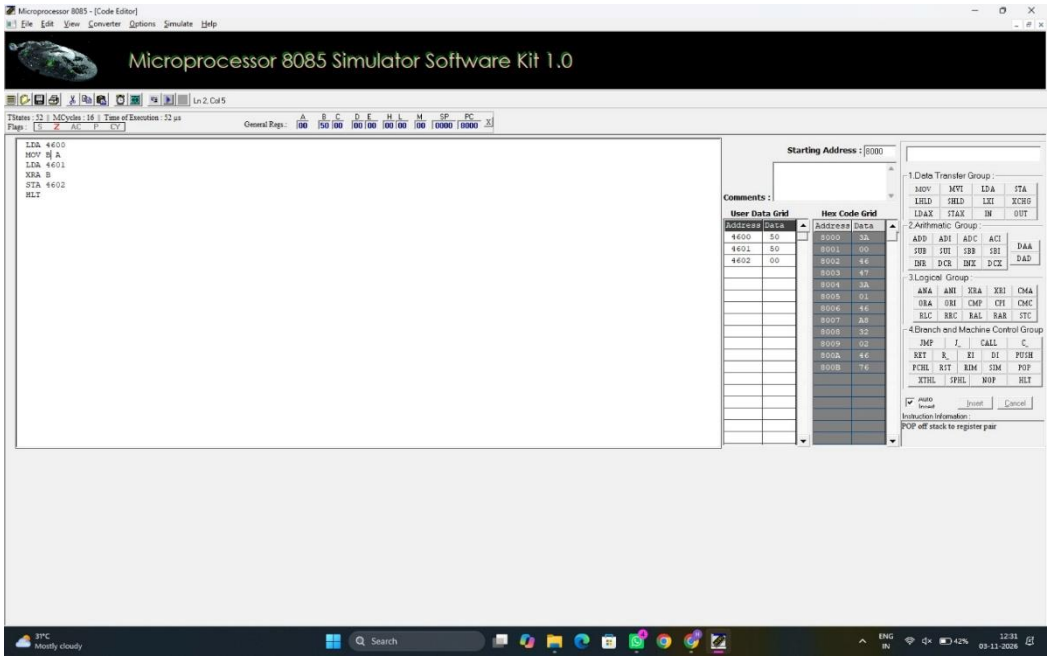
MOV B, A

LDA 4601

XRA B

STA 4602

HLT



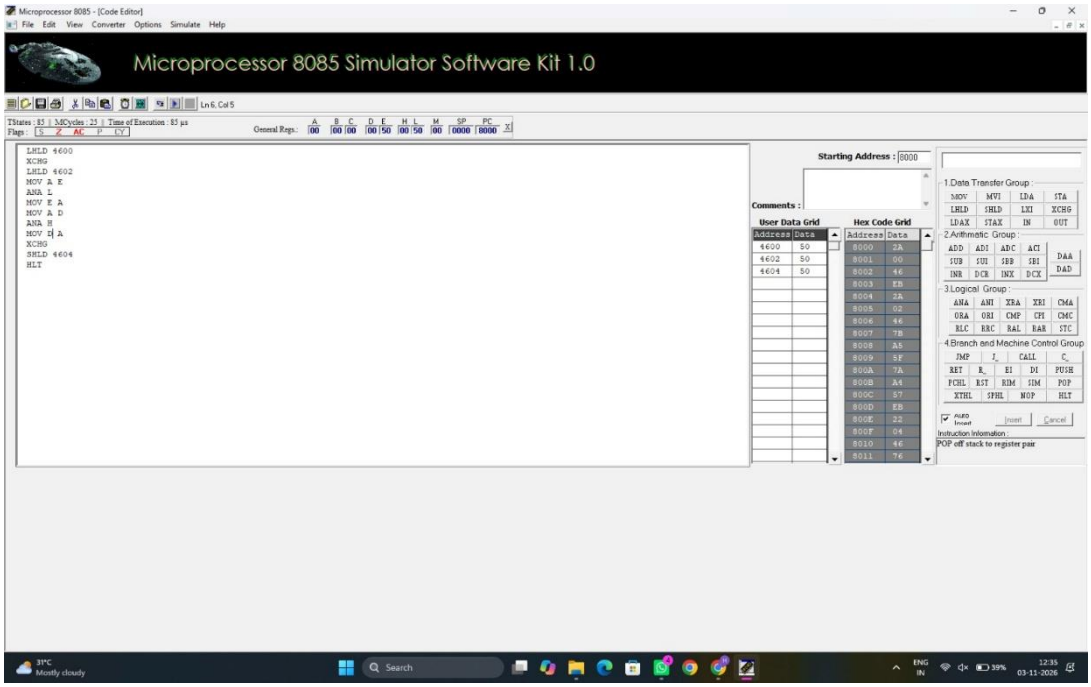
Data Table:

4600	**50**
4601	**50**
4602	**00**

Code:

AND(16 bit):

```
LHLD 4600
XCHG
LHLD 4602
MOV A, E
ANA L
MOV E, A
MOV A, D
ANA H
MOV D, A
XCHG
SHLD 4604
HLT
```



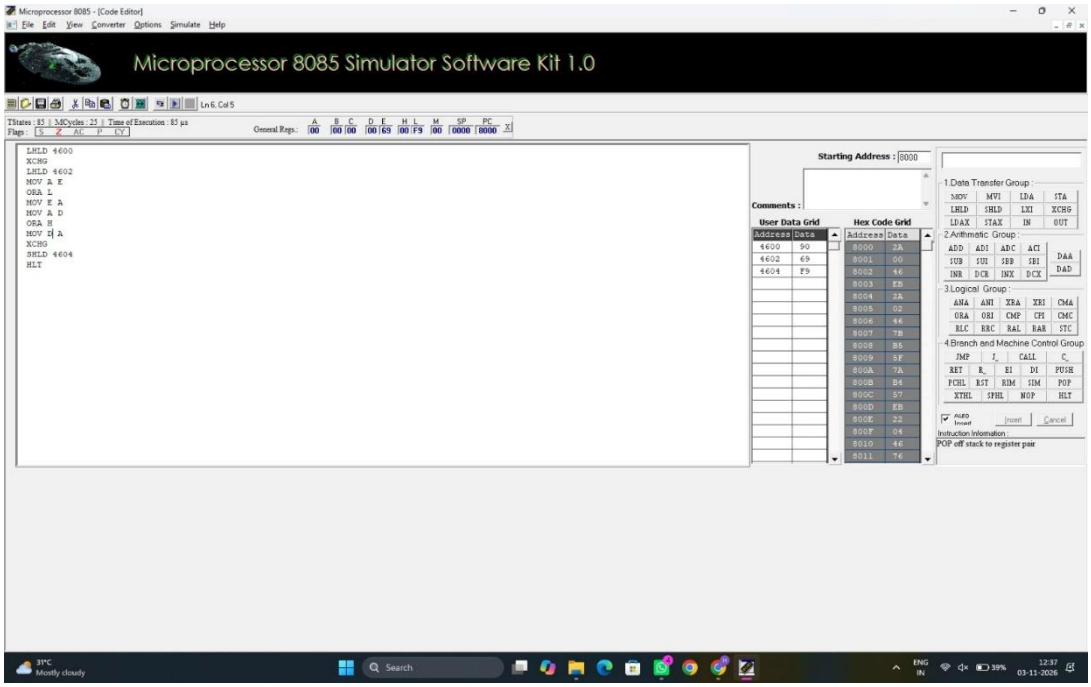
Data Table:

4600	**D5**
4601	**F8**
4602	**CD**
4603	**D5**
4604	**08**
4605	**23**

Code:

OR(16 bit):

```
LHLD 4600
XCHG
LHLD 4602
MOV A, E
ORA L
MOV E, A
MOV A, D
ORA H
MOV D, A
XCHG
SHLD 4604
HLT
```



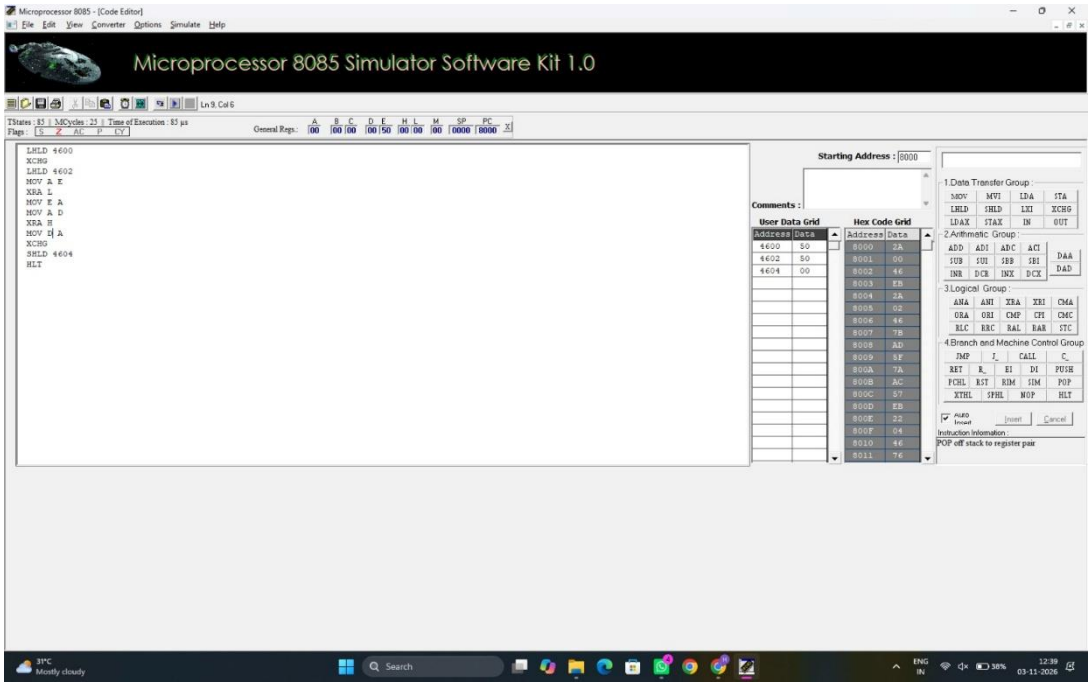
Data Table:

4600	**D5**
4601	**F8**
4602	**CD**
4603	**D5**
4604	**08**
4605	**23**

Code:

XOR(16 bit):

```
LHLD 4600
XCHG
LHLD 4602
MOV A, E
XRA L
MOV E, A
MOV A, D
XRA H
MOV D, A
XCHG
SHLD 4604
HLT
```



Data Table:

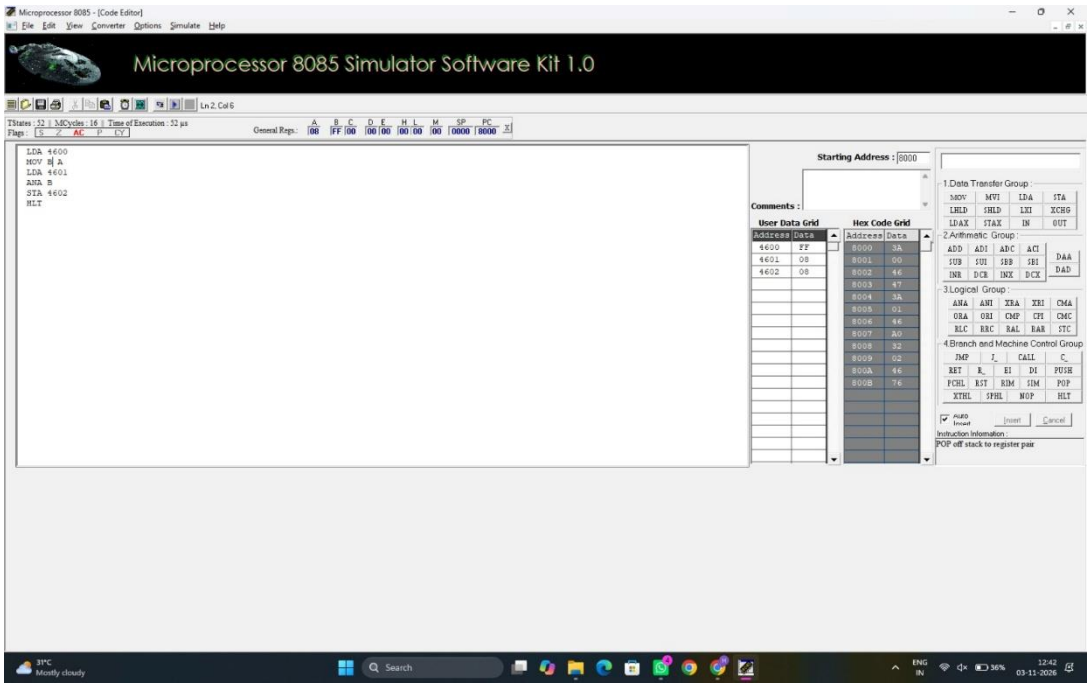
4600	**D5**
4601	**F8**
4602	**CD**
4603	**D5**
4604	**08**
4605	**23**

2. **To analyze and verify Bit Manipulation operations (Masking and Biting).**

Code:

MASK(8 bit):

```
LDA 4600
MOV B, A
LDA 4601
ANA B
STA 4602
HLT
```



Data Table:

4600	**00**
4601	**20**
4602	**20**

Code:

BIT(8 bit):

LDA 4600

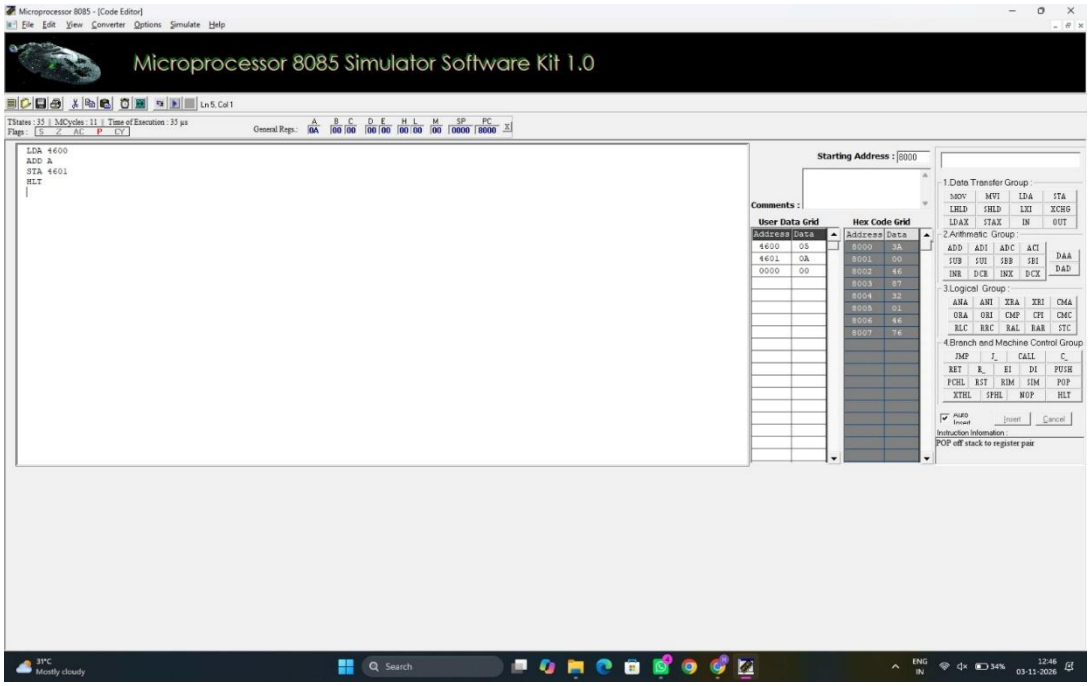
MOV B, A

LDA 4601

ANA B

STA 4602

HLT



Data Table:

4600	**20**
4601	**00**
4602	**00**

Code:

MASK(16 bit):

LHLD 4600

XCHG

LHLD 4602

MOV A, E

ANA L

MOV E, A

MOV A, D

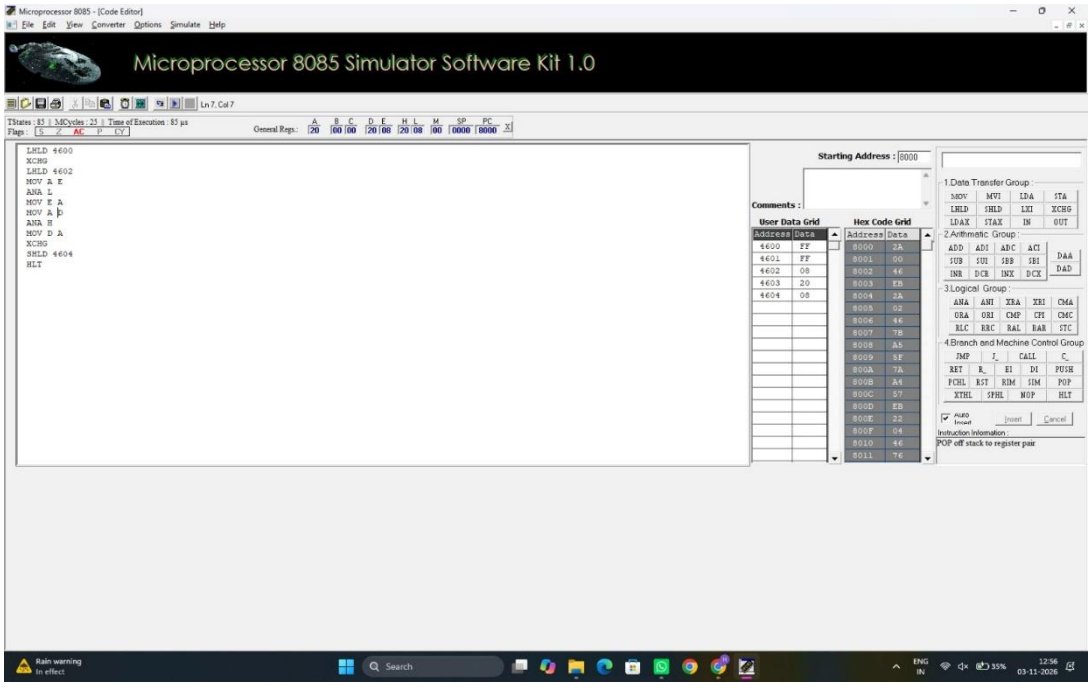
ANA H

MOV D, A

XCHG

SHLD 4604

HLT



Data Table:

4600	**FF**
4601	**FF**
4602	**08**
4603	**20**
4604	**08**

Code:

BIT(16 bit):

LHLD 4600

XCHG

LHLD 4602

MOV A, E

ORA L

MOV E, A

MOV A, D

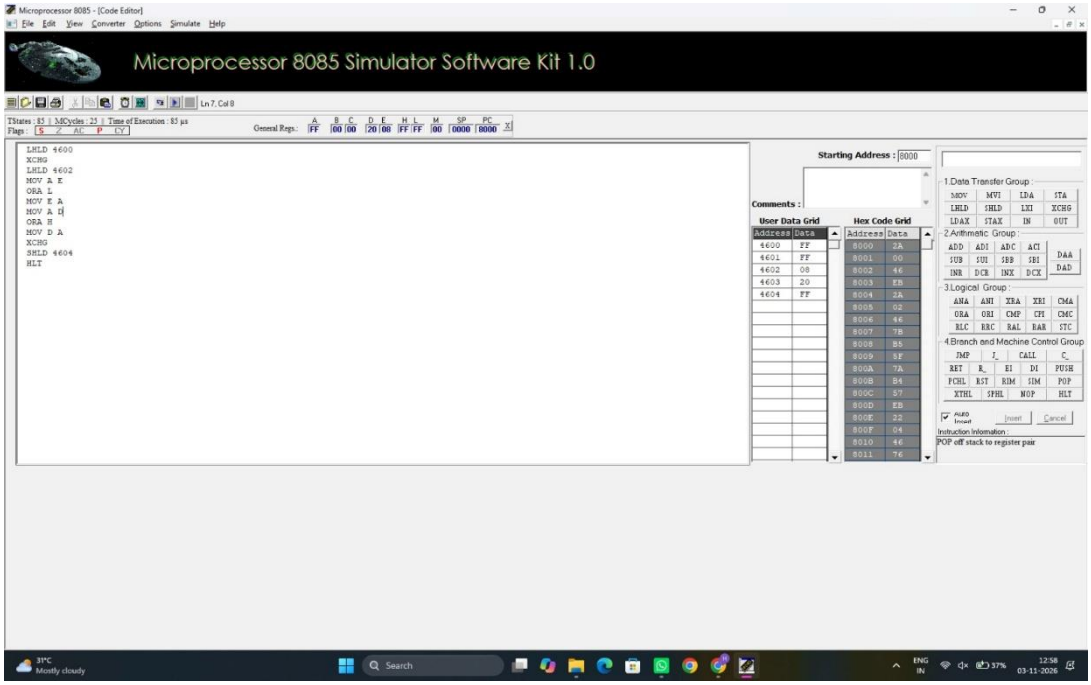
ORA H

MOV D, A

XCHG

SHLD 4604

HLT



Data Table:

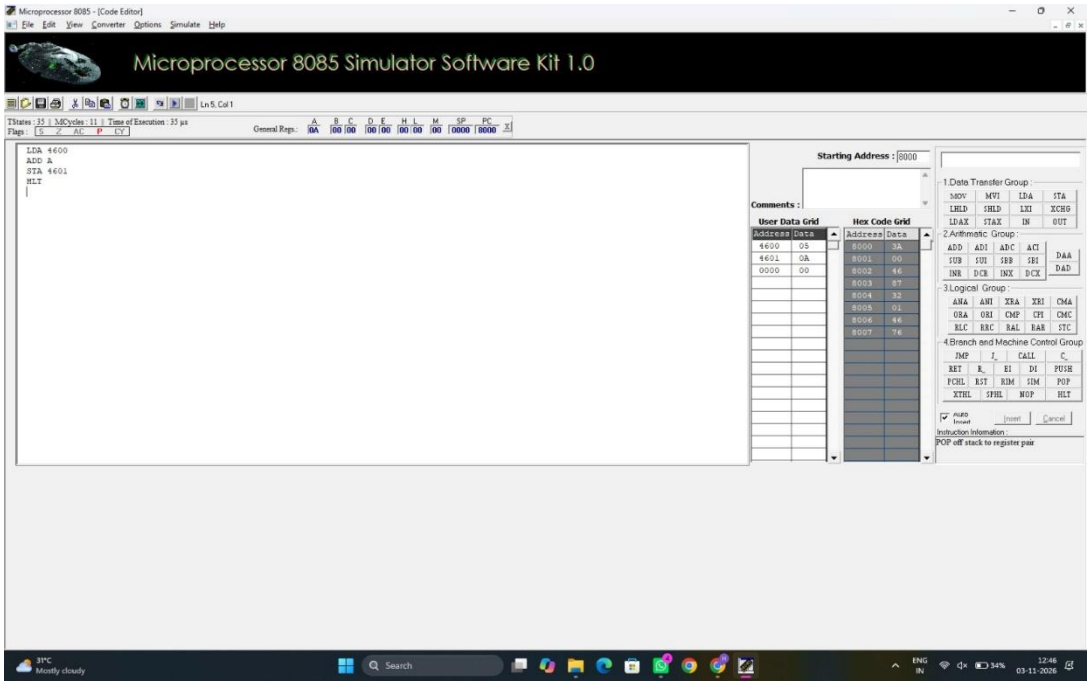
4600	**FF**
4601	**FF**
4602	**08**
4603	**20**
4604	**08**

3. **To analyze and verify Shift and Rotation operations (Logical/Arithmetic shifts left and right) using 8-bit data.**

Code:

Left Shift(8 bit):

```
LDA 4600
ADD A
STA 4601
HLT
```



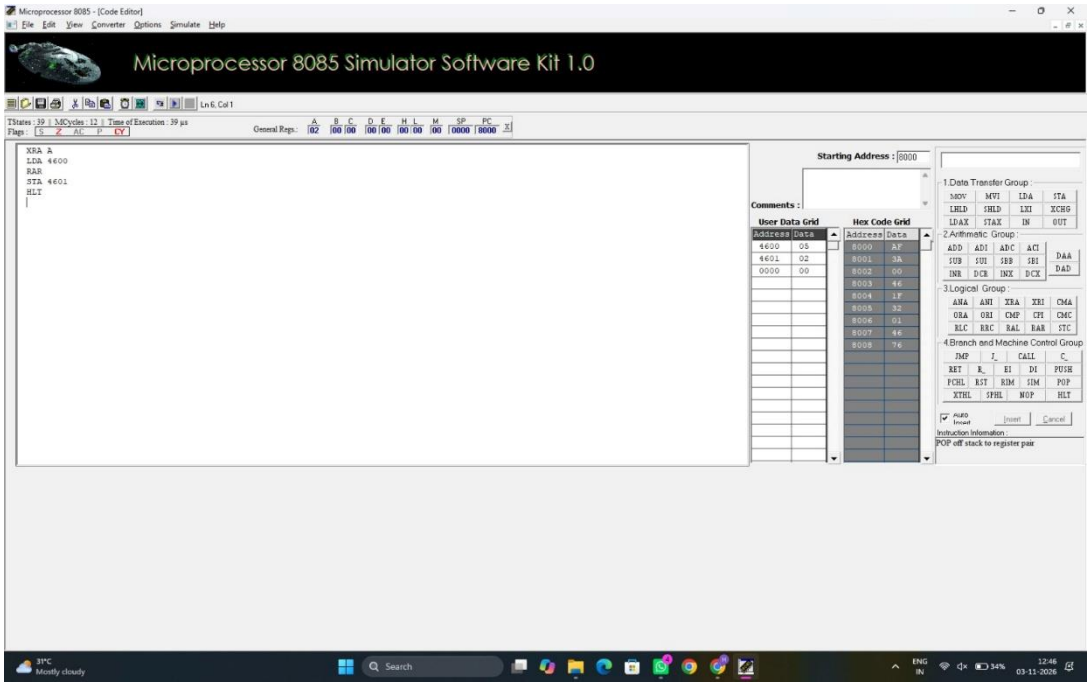
Data Table:

4600	**05**
4601	**0A**
4602	**00**

Code:

Right Shift(8 bit):

```
XRA A
LDA 4600
RAR
STA 4601
HLT
```



Data Table:

4600	**05**
4601	**02**
4602	**00**

Code:

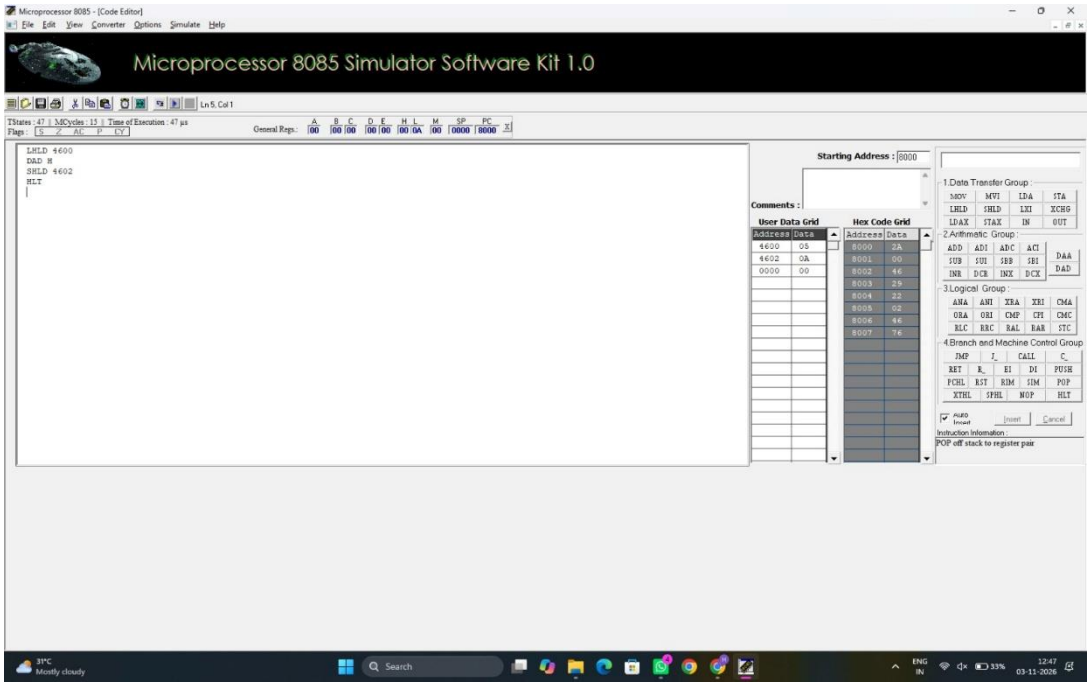
Left Shift(16 bit):

LHLD 4600

DAD H

SHLD 4602

HLT



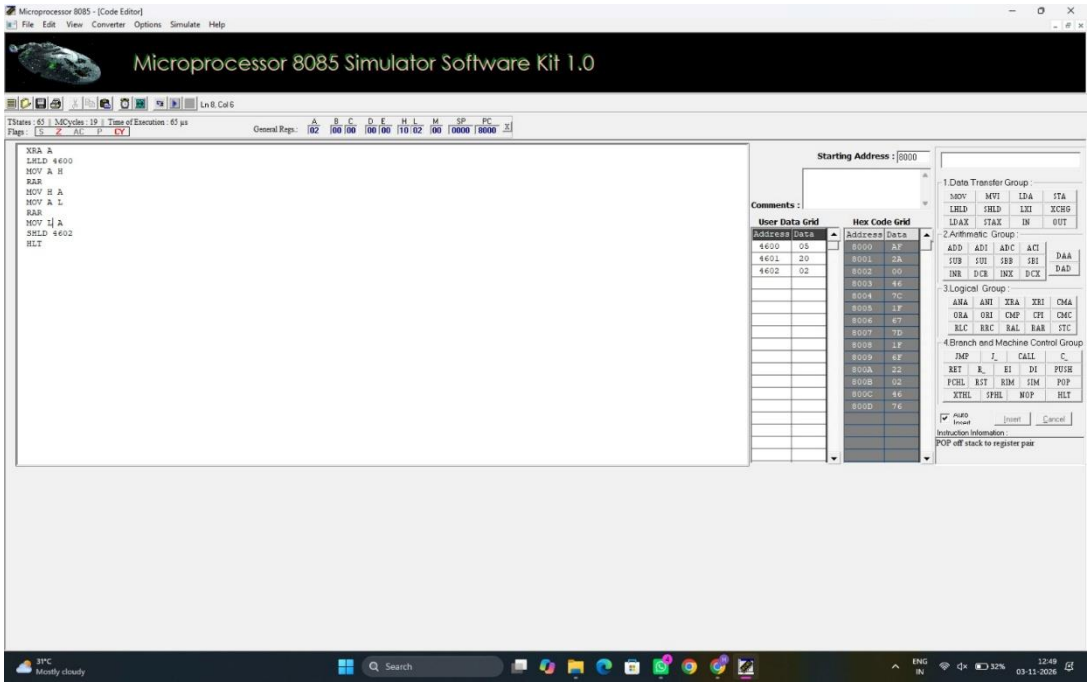
Data Table:

4600	**05**
4601	**0A**
4602	**00**

Code:

Right Shift(16 bit):

```
XRA A
LHLD 4600
MOV A, H
RAR
MOV H, A
MOV A, L
RAR
MOV L, A
SHLD 4602
HLT
```



Data Table:

4600	**05**
4601	**02**
4602	**00**

Result:

1. 8-bit Addition and Subtraction: The target memory location holds the correct 8-bit sum or difference, and the adjacent byte records 01H if an overflow carry or borrow occurred, or 00H if it did not.

2. 16-bit Addition and Subtraction: The sequential memory addresses store the low and high bytes of the correct 16-bit sum or difference, along with a final byte reflecting the state of the terminal carry or borrow flag.

3. 8-bit Multiplication and Division: The output registers or memory addresses contain either a combined 16-bit product from repeated addition, or two distinct 8-bit bytes representing the exact Quotient and Remainder from repeated subtraction.

4. 16-bit Multiplication and Division: The designated target memory blocks accurately hold a multi-byte product (up to 32 bits) generated by the double-addition loop, or a pair of 16-bit values containing the final Quotient and Remainder